

|                          |                                                   |
|--------------------------|---------------------------------------------------|
| Driver management        | Selenium Manager                                  |
| WebDriver isolation      | Thread-local                                      |
| Data source              | CSV                                               |
| CSV organization         | One common CSV                                    |
| CSV library              | OpenCSV                                           |
| CSV storage              | External/local                                    |
| CSV missing              | Fail fast                                         |
| CSV encoding             | UTF-8                                             |
| CSV delimiter            | Comma                                             |
| CSV headers              | Case-insensitive                                  |
| Data execution           | Every CSV row becomes an invocation               |
| Browser/data combination | Same CSV row across browsers                      |
| Test data                | Read-only by contract; mutation warnings selected |
| Environment              | Runtime selection                                 |
| Browser configuration    | TestNG parameter                                  |
| URL                      | TestNG parameter                                  |
| Credentials              | Environment variables                             |
| Sensitive CSV values     | Masked placeholders resolved at runtime           |
| POM                      | One Page Object per page                          |
| Locators                 | By directly inside Page Objects                   |
| Waits                    | Explicit WebDriverWait                            |
| Utilities                | Comprehensive Selenium utilities                  |
| Assertions               | TestNG Assert                                     |
| Authentication           | Username/password                                 |

|                          |                                              |
|--------------------------|----------------------------------------------|
| Login                    | Once per browser session                     |
| Navigation reset         | Return to application home page              |
| Screenshot               | Failure only                                 |
| Reporting                | TestNG default reports                       |
| Logging                  | No dedicated logging framework               |
| Failure classification   | Automation/framework vs application          |
| Configuration validation | At startup                                   |
| Test organization        | Business functionality                       |
| Test categories          | smoke/sanity/regression                      |
| Suite XML                | Separate XML per execution type              |
| Test order               | TestNG priority                              |
| Dependencies             | dependsOnMethods                             |
| Priority                 | Sequential                                   |
| CI/CD                    | Not required currently                       |
| Java                     | <b>Java 21</b>                               |
| Selenium                 | Latest stable Selenium 4 at implementation   |
| TestNG                   | Latest stable compatible TestNG              |
| Maven                    | Maven standard structure                     |
| Naming                   | Standard Java naming                         |
| Project coordinates      | com.automation:selenium-automation-framework |

As of the current implementation date, Selenium's official downloads page lists **Selenium Java 4.46.0 as stable**. TestNG **7.12.0** is available in Maven Central, and the TestNG project documentation lists its versioned update site.

I'll therefore use:

- Java 21

- Selenium 4.46.0
- TestNG 7.12.0
- OpenCSV
- Maven Compiler Plugin 3.15.0

Maven recommends using the compiler release setting rather than old source/target configuration.

---

## 2. Important architecture decisions

There are **three conflicts** in the answers that need to be consciously resolved.

### Conflict 1 — Safari + Windows

Safari cannot run locally on Windows.

Therefore:

Windows

└— Chrome

└— Firefox

└— Edge

macOS

└— Safari

The framework remains browser-independent. Safari simply requires a macOS execution machine.

---

### Conflict 2 — Login once per browser session vs @AfterMethod closes browser

You selected:

- login once per browser session
- @AfterMethod browser cleanup

Those cannot both mean "reuse one browser across multiple test methods."

For the first implementation, I will prioritize **test isolation**:

Test invocation

↓

Create WebDriver

↓

Login

↓

Execute test

↓

Screenshot if failed

↓

Navigate home

↓

Close WebDriver

The `ThreadLocal<WebDriver>` still protects parallel execution.

If later you want true browser-session reuse, we can change the lifecycle to `@AfterClass`/session management.

---

### Conflict 3 — Shared mutable `String[]`

You explicitly selected shared mutable test data and mutation warnings.

That is **not my recommended production practice**.

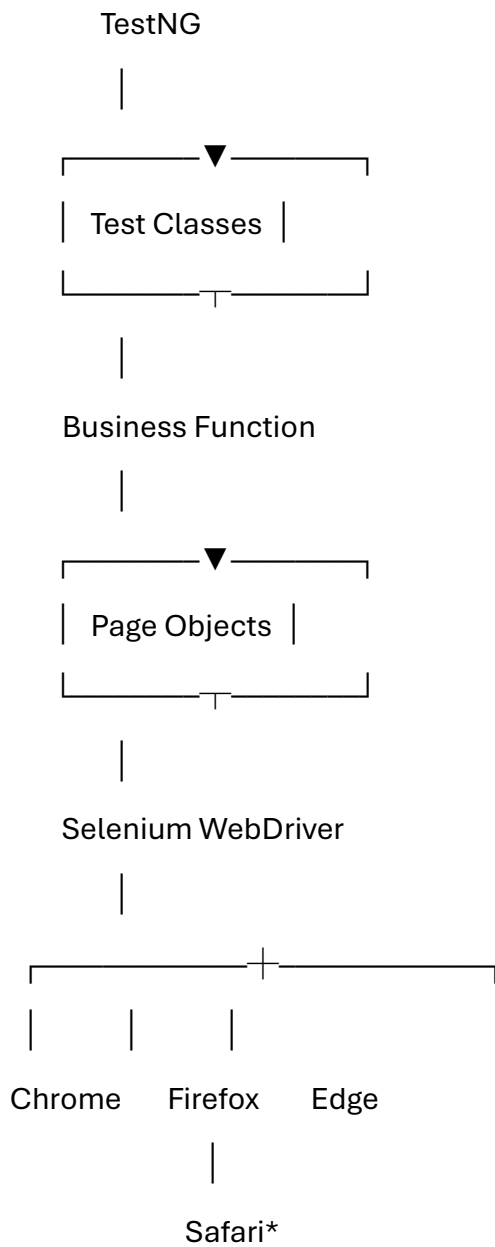
The framework will therefore:

1. Treat CSV source data as read-only.
2. Detect/compare mutations.
3. Log a warning using `System.err` because you selected no dedicated logging framework.
4. Never modify the original CSV file.

---

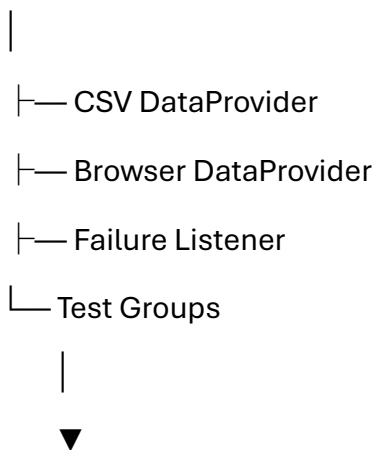
## 3. Architecture

The framework will follow:



Supporting layers:

TestNG



## Framework

- └─ DriverFactory
- └─ DriverManager
- └─ Config
- └─ CSV Reader
- └─ Wait Utilities
- └─ Browser Utilities
- └─ Screenshot Utility

---

## 4. Recommended project structure

selenium-automation-framework/

- |
- └─ pom.xml
- |
- └─ README.md
- |
- └─ src/
  - | |
  - | └─ main/
    - | | └─ java/
      - | | └─ com/
        - | | └─ automation/
          - | | └─ config/
            - | | └─ ConfigManager.java
          - | | └─ driver/
            - | | └─ DriverFactory.java

```
| | | | └─ DriverManager.java
| | | |
| | | | └─ pages/
| | | | | └─ LoginPage.java
| | | | | └─ HomePage.java
| | | | |
| | | | └─ utils/
| | | | | └─ WaitUtils.java
| | | | | └─ BrowserUtils.java
| | | | | └─ ScreenshotUtils.java
| | | |
| | | | └─ test/
| | | | |
| | | | | └─ java/
| | | | | | └─ com/
| | | | | | | └─ automation/
| | | | | | | | └─ dataproviders/
| | | | | | | | | └─ CsvDataProvider.java
| | | | | | | | | └─ BrowserDataProvider.java
| | | | | | | |
| | | | | | | └─ listeners/
| | | | | | | | └─ TestFailureListener.java
| | | | | | |
| | | | | | | └─ tests/
| | | | | | | | └─ LoginTest.java
| | | | |
| | | | | └─ resources/
| | | | |
| | | | | └─ suites/
```

```
|      |— smoke.xml
|      |— sanity.xml
|      └— regression.xml
|
└— external-test-data/
    └— LoginTest.csv
```

The important distinction is:

src/main/java

Framework implementation

src/test/java

Actual automated tests

external-test-data

Business test data

That keeps the framework reusable.

---

## 5. pom.xml

Selenium's official documentation uses the selenium-java Maven dependency, and Selenium Manager is part of the Selenium ecosystem, so we don't need WebDriverManager.

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="
    http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
```

<modelVersion>4.0.0</modelVersion>

<groupId>com.automation</groupId>

<artifactId>selenium-automation-framework</artifactId>

<version>1.0.0</version>

<properties>

<project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

<maven.compiler.release>21</maven.compiler.release>

<selenium.version>4.46.0</selenium.version>

<testng.version>7.12.0</testng.version>

<opencsv.version>5.12.0</opencsv.version>

<maven.compiler.plugin.version>3.15.0</maven.compiler.plugin.version>

<maven.surefire.version>3.5.3</maven.surefire.version>

</properties>

<dependencies>

<!-- Selenium -->

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>${selenium.version}</version>
</dependency>
```

```
<!-- TestNG -->
```

```
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>${testng.version}</version>
  <scope>test</scope>
</dependency>
```

```
<!-- CSV parser -->
```

```
<dependency>
  <groupId>com.opencsv</groupId>
  <artifactId>opencsv</artifactId>
  <version>${opencsv.version}</version>
</dependency>
```

```
</dependencies>
```

```
<build>
```

```
<plugins>
```

```
  <!-- Java compiler -->
```

```

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <version>${maven.compiler.plugin.version}</version>
  <configuration>
    <release>21</release>
  </configuration>
</plugin>

<!-- TestNG execution -->
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>${maven.surefire.version}</version>
</plugin>

</plugins>

</build>

</project>

```

---

## 6. Configuration Manager

Because you selected **TestNG parameters**, configuration values will come from TestNG rather than a properties file.

We still need a centralized class to hold validated values.

```
package com.automation.config;
```

```
public final class ConfigManager {

    private static String browser;

    private static String baseUrl;

    private ConfigManager() {
        // Prevent object creation.
    }

    public static void initialize(String browser, String baseUrl) {

        if (browser == null || browser.isBlank()) {
            throw new IllegalArgumentException(
                "Browser parameter is mandatory."
            );
        }

        if (baseUrl == null || baseUrl.isBlank()) {
            throw new IllegalArgumentException(
                "Base URL parameter is mandatory."
            );
        }

        ConfigManager.browser = browser.toLowerCase();

        ConfigManager.baseUrl = baseUrl;
    }

    public static String getBrowser() {
```

```
        return browser;
    }

    public static String getBaseUrl() {
        return baseUrl;
    }
}
```

---

## 7. DriverManager

This is the most important class for parallel execution.

Each TestNG thread receives its own WebDriver.

```
package com.automation.driver;
```

```
import org.openqa.selenium.WebDriver;
```

```
public final class DriverManager {

    private DriverManager() {
    }

    private static final ThreadLocal<WebDriver> DRIVER =
        new ThreadLocal<>();

    public static void setDriver(WebDriver driver) {
        DRIVER.set(driver);
    }

    public static WebDriver getDriver() {
```

```
WebDriver driver = DRIVER.get();
```

```
if (driver == null) {
```

```
    throw new IllegalStateException(
```

```
        "WebDriver has not been initialized for this thread."
```

```
    );
```

```
}
```

```
return driver;
```

```
}
```

```
public static void quitDriver() {
```

```
    WebDriver driver = DRIVER.get();
```

```
    if (driver != null) {
```

```
        try {
```

```
            driver.quit();
```

```
        } finally {
```

```
            DRIVER.remove();
```

```
        }
```

```
    }
```

```
}
```

```
}
```

**Why ThreadLocal?**

Without it:

Thread 1 —┐

Thread 2 —┴—> Same WebDriver ❌

Thread 3 —┘

With it:

Thread 1 —> Driver 1

Thread 2 —> Driver 2

Thread 3 —> Driver 3

Thread 4 —> Driver 4

That is essential for parallel Selenium execution.

---

## 8. DriverFactory

Selenium Manager automatically handles driver discovery.

```
package com.automation.driver;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.openqa.selenium.edge.EdgeDriver;
```

```
import org.openqa.selenium.firefox.FirefoxDriver;
```

```
import org.openqa.selenium.safari.SafariDriver;
```

```
public final class DriverFactory {
```

```
    private DriverFactory() {
```

```
    }
```

```
    public static WebDriver createDriver(String browser) {
```

```

    if (browser == null || browser.isBlank()) {
        throw new IllegalArgumentException(
            "Browser cannot be null or empty."
        );
    }

    return switch (browser.toLowerCase()) {

        case "chrome" -> new ChromeDriver();

        case "firefox" -> new FirefoxDriver();

        case "edge" -> new EdgeDriver();

        case "safari" -> new SafariDriver();

        default -> throw new IllegalArgumentException(
            "Unsupported browser: " + browser
        );
    };
}

```

No:

System.setProperty(...)

and no:

WebDriverManager.chromedriver().setup();

are required.

---

## 9. Wait utility

You selected explicit waits.

```
package com.automation.utils;

import java.time.Duration;

import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.WebElement;
import org.openqa.selenium.support.ui.ExpectedConditions;
import org.openqa.selenium.support.ui.WebDriverWait;

public final class WaitUtils {

    private static final Duration DEFAULT_TIMEOUT =
        Duration.ofSeconds(15);

    private WaitUtils() {
    }

    public static WebElement waitForVisible(
        WebDriver driver,
        By locator) {

        return new WebDriverWait(
            driver,
            DEFAULT_TIMEOUT
        ).until(
```

```
        ExpectedConditions.visibilityOfElementLocated(locator)
    );
}
```

```
public static WebElement waitForClickable(
    WebDriver driver,
    By locator) {

    return new WebDriverWait(
        driver,
        DEFAULT_TIMEOUT
    ).until(
        ExpectedConditions.elementToBeClickable(locator)
    );
}
```

```
public static boolean waitForUrlContains(
    WebDriver driver,
    String value) {

    return new WebDriverWait(
        driver,
        DEFAULT_TIMEOUT
    ).until(
        ExpectedConditions.urlContains(value)
    );
}
}
```

We deliberately avoid:

```
Thread.sleep(5000);
```

because fixed sleeps don't synchronize with application state.

---

## **10. Browser utilities**

```
package com.automation.utils;
```

```
import org.openqa.selenium.WebDriver;
```

```
public final class BrowserUtils {
```

```
    private BrowserUtils() {  
    }  
}
```

```
    public static void maximize(WebDriver driver) {  
        driver.manage().window().maximize();  
    }  
}
```

```
    public static void navigateTo(  
        WebDriver driver,  
        String url) {  
  
        driver.get(url);  
    }  
}
```

```
    public static void navigateHome(  
        WebDriver driver,  
        String homeUrl) {
```

```
        driver.navigate().to(homeUrl);
    }
}
```

You selected the browser's default window size, so we **do not call** `maximize()` in the framework lifecycle.

The utility remains available if a particular test needs it.

---

## 11. Screenshot utility

Screenshots are captured only when the test fails.

```
package com.automation.utils;

import java.io.File;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.StandardCopyOption;

import org.openqa.selenium.OutputType;
import org.openqa.selenium.TakesScreenshot;
import org.openqa.selenium.WebDriver;

public final class ScreenshotUtils {

    private ScreenshotUtils() {
    }

    public static Path capture(
        WebDriver driver,
```

```
String testName) {

try {

    Path directory =
        Path.of("test-output", "screenshots");

    Files.createDirectories(directory);

    String fileName =
        testName + "_" +
        System.currentTimeMillis() +
        ".png";

    Path destination =
        directory.resolve(fileName);

    File source =
        ((TakesScreenshot) driver)
            .getScreenshotAs(OutputType.FILE);

    Files.copy(
        source.toPath(),
        destination,
        StandardCopyOption.REPLACE_EXISTING
    );

    return destination;
}
```

```
    } catch (Exception e) {

        System.err.println(
            "Unable to capture screenshot: "
            + e.getMessage()
        );

        return null;
    }
}
}
```

---

## 12. CSV Reader

OpenCSV handles parsing.

```
package com.automation.dataproviders;
```

```
import com.opencsv.CSVReader;
```

```
import com.opencsv.exceptions.CsvValidationException;
```

```
import java.io.FileReader;
```

```
import java.io.IOException;
```

```
import java.util.ArrayList;
```

```
import java.util.List;
```

```
public final class CsvDataProvider {
```

```
    private CsvDataProvider() {
```

```
}
```

```
public static List<String[]> readCsv(  
    String filePath) {
```

```
    List<String[]> rows = new ArrayList<>();
```

```
    try (CSVReader reader =  
        new CSVReader(  
            new FileReader(filePath))) {
```

```
        String[] headers = reader.readNext();
```

```
        if (headers == null) {  
            throw new IllegalArgumentException(  
                "CSV file is empty: " + filePath  
            );  
        }
```

```
        String[] row;
```

```
        while ((row = reader.readNext()) != null) {  
            rows.add(row);  
        }
```

```
    } catch (IOException | CsvValidationException e) {
```

```
        throw new RuntimeException(  
            e.getMessage(), e);  
    }
```

```
        "Unable to read CSV: " + filePath,  
        e  
    );  
}  
  
return rows;  
}  
}
```

---

### 13. Browser DataProvider

You selected:

same CSV row × multiple browsers

So we create the Cartesian product:

CSV rows

×

Browsers

Example:

TC001 + Chrome

TC001 + Firefox

TC001 + Edge

TC001 + Safari

TC002 + Chrome

TC002 + Firefox

TC002 + Edge

TC002 + Safari

package com.automation.dataproviders;

```
import org.testng.annotations.DataProvider;

import java.util.ArrayList;
import java.util.List;

public final class BrowserDataProvider {

    private BrowserDataProvider() {
    }

    @DataProvider(
        name = "browserAndCsvData",
        parallel = true
    )
    public static Object[][] browserAndCsvData() {

        String csvPath =
            System.getenv("AUTOMATION_CSV_PATH");

        if (csvPath == null || csvPath.isBlank()) {
            throw new IllegalStateException(
                "AUTOMATION_CSV_PATH environment variable "
                + "is mandatory."
            );
        }

        List<String[]> csvRows =
            CsvDataProvider.readCsv(csvPath);
    }
}
```

```

String[] browsers = {
    "chrome",
    "firefox",
    "edge",
    "safari"
};

List<Object[]> executionData =
    new ArrayList<>();

for (String[] row : csvRows) {

    for (String browser : browsers) {

        executionData.add(
            new Object[]{
                browser,
                row
            }
        );
    }
}

return executionData.toArray(
    new Object[0][]
);
}

```

```
}
```

---

## 14. Base Page

Although you selected one Page Object per page, a very small technical base class avoids duplicating the WebDriver reference.

```
package com.automation.pages;
```

```
import org.openqa.selenium.WebDriver;
```

```
public abstract class BasePage {
```

```
    protected final WebDriver driver;
```

```
    protected BasePage(WebDriver driver) {
```

```
        this.driver = driver;
```

```
    }
```

```
}
```

This is **not** a business workflow layer.

---

## 15. Login Page

```
package com.automation.pages;
```

```
import com.automation.utils.WaitUtils;
```

```
import org.openqa.selenium.By;
```

```
import org.openqa.selenium.WebDriver;
```

```
public class LoginPage extends BasePage {
```

```
private final By username =  
    By.id("username");
```

```
private final By password =  
    By.id("password");
```

```
private final By loginButton =  
    By.id("loginButton");
```

```
public LoginPage(WebDriver driver) {  
    super(driver);  
}
```

```
public void enterUsername(String value) {  
  
    WaitUtils.waitForVisible(  
        driver,  
        username  
    ).sendKeys(value);  
}
```

```
public void enterPassword(String value) {  
  
    WaitUtils.waitForVisible(  
        driver,  
        password  
    ).sendKeys(value);  
}
```

```
}
```

```
public void clickLogin() {
```

```
    WaitUtils.waitForClickable(
```

```
        driver,
```

```
        loginButton
```

```
    ).click();
```

```
}
```

```
public HomePage login(
```

```
    String usernameValue,
```

```
    String passwordValue) {
```

```
    enterUsername(usernameValue);
```

```
    enterPassword(passwordValue);
```

```
    clickLogin();
```

```
    return new HomePage(driver);
```

```
}
```

```
}
```

The locators above are placeholders and should be replaced with your application's actual locators.

---

## 16. Home Page

```
package com.automation.pages;
```

```
import com.automation.utils.WaitUtils;
```

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;

public class HomePage extends BasePage {

    private final By homeHeader =
        By.id("homeHeader");

    public HomePage(WebDriver driver) {
        super(driver);
    }

    public boolean isDisplayed() {

        return WaitUtils
            .waitForVisible(driver, homeHeader)
            .isDisplayed();
    }
}
```

---

## 17. Failure listener

Because you selected:

- TestNG default reports
- screenshot on failure
- browser/page details
- failure classification

we can use a TestNG listener.

```
package com.automation.listeners;

import com.automation.driver.DriverManager;
import com.automation.utils.ScreenshotUtils;

import org.openqa.selenium.WebDriver;
import org.testng.ITestListener;
import org.testng.ITestResult;

public class TestFailureListener
    implements ITestListener {

    @Override
    public void onTestFailure(
        ITestResult result) {

        try {

            WebDriver driver =
                DriverManager.getDriver();

            String testName =
                result.getMethod()
                    .getMethodName();

            System.err.println(
                "TEST FAILURE"
            );
        }
    }
}
```

```
System.err.println(  
    "Test: " + testName  
);
```

```
System.err.println(  
    "URL: " + driver.getCurrentUrl()  
);
```

```
System.err.println(  
    "Title: " + driver.getTitle()  
);
```

```
System.err.println(  
    "Failure Type: "  
        + classifyFailure(  
            result.getThrowable()  
        )  
);
```

```
ScreenshotUtils.capture(  
    driver,  
    testName  
);
```

```
} catch (Exception e) {
```

```
System.err.println(  
    "Failure listener error: "
```

```
        + e.getMessage()
    );
}
}
```

```
private String classifyFailure(
    Throwable throwable) {
```

```
    if (throwable == null) {
        return "UNKNOWN";
    }
```

```
    String name =
        throwable.getClass().getSimpleName();
```

```
    if (name.contains("Timeout")) {
        return "AUTOMATION/SYNCHRONIZATION";
    }
```

```
    if (name.contains("NoSuchElement")) {
        return "AUTOMATION/LOCATOR";
    }
```

```
    if (name.contains("WebDriver")) {
        return "AUTOMATION/WEBDRIVER";
    }
```

```
    if (name.contains("Assertion")) {
```

```
        return "APPLICATION/TEST_VALIDATION";
    }

    return "APPLICATION_OR_AUTOMATION";
}
}
```

---

## 18. Test class

You selected setup/teardown at the test-class level rather than a centralized BaseTest.

So the lifecycle is explicit here.

```
package com.automation.tests;
```

```
import com.automation.config.ConfigManager;
import com.automation.driver.DriverFactory;
import com.automation.driver.DriverManager;
import com.automation.listeners.TestFailureListener;
import com.automation.pages.HomePage;
import com.automation.pages.LoginPage;
import com.automation.utils.BrowserUtils;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.testng.Assert;
import org.testng.annotations.AfterMethod;
import org.testng.annotations.BeforeMethod;
import org.testng.annotations.Listeners;
import org.testng.annotations.Parameters;
import org.testng.annotations.Test;
```

```
@Listeners(TestFailureListener.class)

public class LoginTest {

    @BeforeMethod
    @Parameters({"browser", "baseUrl"})
    public void setUp(
        String browser,
        String baseUrl) {

        ConfigManager.initialize(
            browser,
            baseUrl
        );

        WebDriver driver =
            DriverFactory.createDriver(browser);

        DriverManager.setDriver(driver);

        BrowserUtils.navigateTo(
            driver,
            baseUrl
        );
    }

    @Test(
        priority = 1,
```

```
        groups = {"smoke", "sanity", "regression"}  
    )  
    public void verifyValidLogin() {
```

```
        WebDriver driver =  
            DriverManager.getDriver();
```

```
        String username =  
            System.getenv(  
                "AUTOMATION_USERNAME"  
            );
```

```
        String password =  
            System.getenv(  
                "AUTOMATION_PASSWORD"  
            );
```

```
        Assert.assertNotNull(  
            username,  
            "AUTOMATION_USERNAME is missing."  
        );
```

```
        Assert.assertNotNull(  
            password,  
            "AUTOMATION_PASSWORD is missing."  
        );
```

```
        LoginPage loginPage =
```

```
new LoginPage(driver);
```

```
HomePage homePage =
```

```
    loginPage.login(
```

```
        username,
```

```
        password
```

```
    );
```

```
Assert.assertTrue(
```

```
    homePage.isDisplayed(),
```

```
    "Home page should be displayed after login."
```

```
);
```

```
}
```

```
@AfterMethod
```

```
public void tearDown() {
```

```
    try {
```

```
        WebDriver driver =
```

```
            DriverManager.getDriver();
```

```
        /*
```

```
        * Return to home page before closing.
```

```
        * This implements the selected browser-state
```

```
        * reset behavior.
```

```
        */
```

```
    } catch {
```

```

        BrowserUtils.navigateHome(
            driver,
            ConfigManager.getBaseUrl()
        );
    } catch (Exception e) {
        System.err.println(
            "Unable to navigate home during cleanup: "
            + e.getMessage()
        );
    }

} finally {

    /*
     * Always terminate the browser and remove
     * the ThreadLocal reference.
     */
    DriverManager.quitDriver();
}
}
}

```

---

## 19. Important improvement to browser parameterization

There is a design issue in the choices we gathered:

You selected both:

```
<parameter name="browser" value="chrome"/>
```

and:

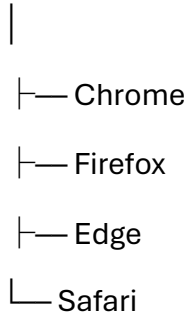
```
@DataProvider(parallel = true)
```

for multiple browsers.

TestNG parameters are **single configuration values**, while the DataProvider naturally produces multiple values.

Therefore, the clean implementation is:

DataProvider



rather than trying to pass:

`browser="chrome,firefox,edge,safari"`

into a normal TestNG parameter.

This is one of the areas where the architecture should override an earlier questionnaire answer because the later requirement — **parallel browser matrix** — is more specific.

---

## 20. TestNG suite

For the smoke suite:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE suite SYSTEM
```

```
    "https://testng.org/testng-1.0.dtd">
```

```
<suite name="Smoke Suite"
```

```
    parallel="methods"
```

```
    thread-count="4">
```

```
    <test name="Smoke Tests">
```

```
<parameter
  name="baseUrl"
  value="https://your-application-url.com"/>
```

```
<classes>
```

```
  <class name="com.automation.tests.LoginTest"/>
```

```
</classes>
```

```
</test>
```

```
</suite>
```

Sanity:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE suite SYSTEM
```

```
  "https://testng.org/testng-1.0.dtd">
```

```
<suite name="Sanity Suite"
```

```
  parallel="methods"
```

```
  thread-count="4">
```

```
<test name="Sanity Tests">
```

```
<parameter
```

```
  name="baseUrl"
```

value="https://your-application-url.com"/>

<classes>

<class name="com.automation.tests.LoginTest"/>

</classes>

</test>

</suite>

Regression:

<?xml version="1.0" encoding="UTF-8"?>

<!DOCTYPE suite SYSTEM

"https://testng.org/testng-1.0.dtd">

<suite name="Regression Suite"

parallel="methods"

thread-count="4">

<test name="Regression Tests">

<parameter

name="baseUrl"

value="https://your-application-url.com"/>

<classes>

```
<class name="com.automation.tests.LoginTest"/>
```

```
</classes>
```

```
</test>
```

```
</suite>
```

---

## 21. CSV structure

Because you selected **one common CSV**, I'd use:

TestId,Username,Password,ExpectedResult

TC\_LOGIN\_001,validuser,{AUTOMATION\_PASSWORD},SUCCESS

TC\_LOGIN\_002,invaliduser,{AUTOMATION\_PASSWORD},FAILURE

The important point is that:

TestId

is used for report identification.

For example:

verifyValidLogin - TC\_LOGIN\_001

rather than just:

verifyValidLogin

---

## 22. Sensitive data

Your selected strategy is:

CSV

|

|— normal test data

|

└─ \${AUTOMATION\_PASSWORD}

|



Environment variable

Windows example:

```
$env:AUTOMATION_USERNAME="testuser"
```

```
$env:AUTOMATION_PASSWORD="secret"
```

```
$env:AUTOMATION_CSV_PATH="C:\automation\data>LoginTest.csv"
```

Then:

```
System.getenv("AUTOMATION_PASSWORD");
```

The password should **never** be committed to Git.

---

## 23. Execution

Compile:

```
mvn clean test-compile
```

Run smoke:

```
mvn test -Dsurefire.suiteXmlFiles=src/test/resources/suites/smoke.xml
```

Run sanity:

```
mvn test -Dsurefire.suiteXmlFiles=src/test/resources/suites/sanity.xml
```

Run regression:

```
mvn test -Dsurefire.suiteXmlFiles=src/test/resources/suites/regression.xml
```

Selenium's current Java examples likewise use Maven to compile and execute Selenium tests.

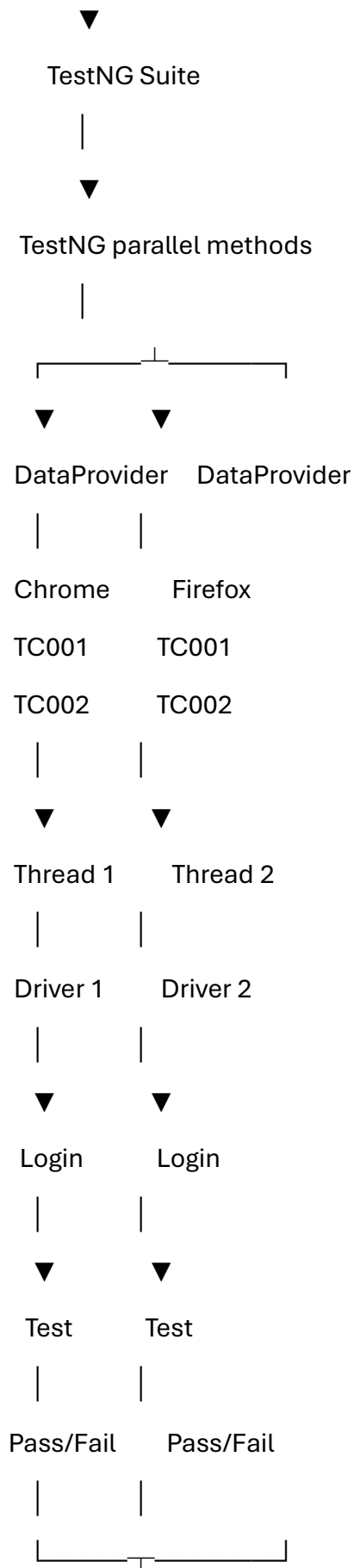
---

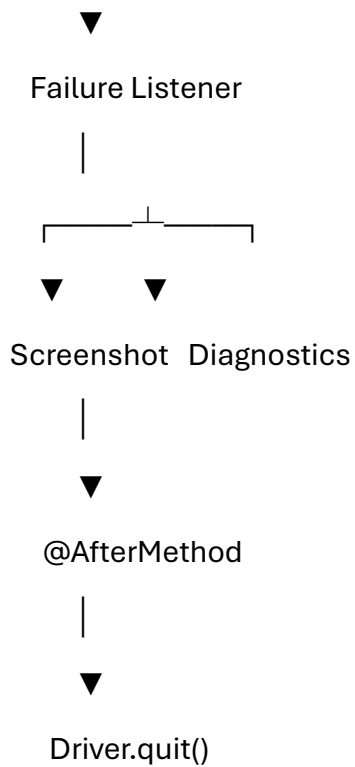
## 24. Execution flow

Your final framework will work like this:

Maven

|





---

## 25. Why this architecture is scalable

The important principle is **separation of responsibility**.

### Test class

Responsible for:

What should be tested?

### Page Object

Responsible for:

How do I interact with the page?

### DriverFactory

Responsible for:

Which browser should I create?

### DriverManager

Responsible for:

Which driver belongs to this thread?

### DataProvider

Responsible for:

Which data should this test execute with?

### **WaitUtils**

Responsible for:

How should Selenium synchronize?

### **Listener**

Responsible for:

What happens when something fails?

That separation prevents your test classes from becoming giant automation classes.

---

## **26. What I would NOT add**

Because you specifically requested an industry-grade but scalable framework, it's equally important not to over-engineer it.

I would **not initially add**:

- Cucumber
- Spring
- Guice
- ExtentReports
- Allure
- Log4j
- WebDriverManager
- Selenium Grid
- API framework
- Database layer
- Docker
- cloud browser providers
- complicated custom annotations
- huge generic utility libraries

You don't currently need them.

Selenium Manager already addresses driver management, while TestNG provides suite organization, groups, parallel execution and reporting.

---

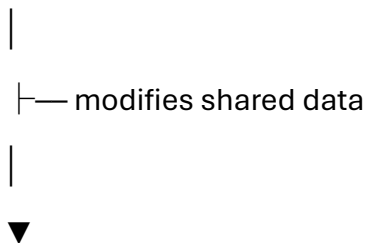
## 27. One change I strongly recommend

There is one requirement I would **not carry into a production framework unchanged**:

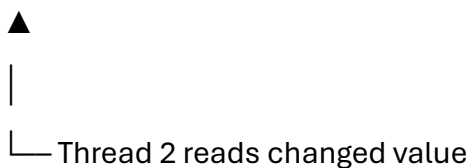
Shared mutable CSV objects between parallel tests.

Your selected approach can cause:

Thread 1

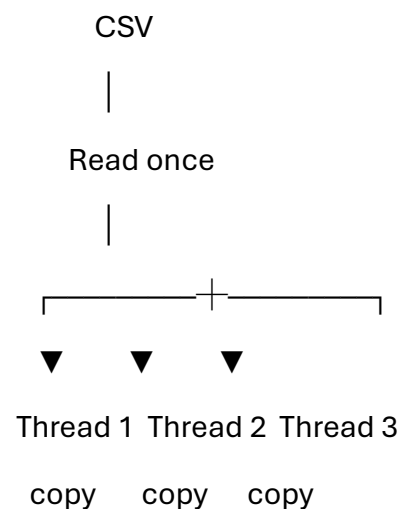


Shared String[]



That can produce extremely difficult-to-debug failures.

The production-safe design should instead be:



The CSV itself remains read-only.

That gives you **parallelism without shared mutable state**.

---

## **28. Architecture maturity path**

The framework we've designed can evolve without rewriting the Page Objects.

### **Phase 1 — Current**

Local

+

Selenium Manager

+

TestNG

+

CSV

+

ThreadLocal

### **Phase 2**

Jenkins / CI

+

Headless execution

+

Historical reports

### **Phase 3**

Selenium Grid

+

Distributed browsers

### **Phase 4**

Cloud execution

+

Browser/device matrix

## Phase 5

API + UI

+

Database validation

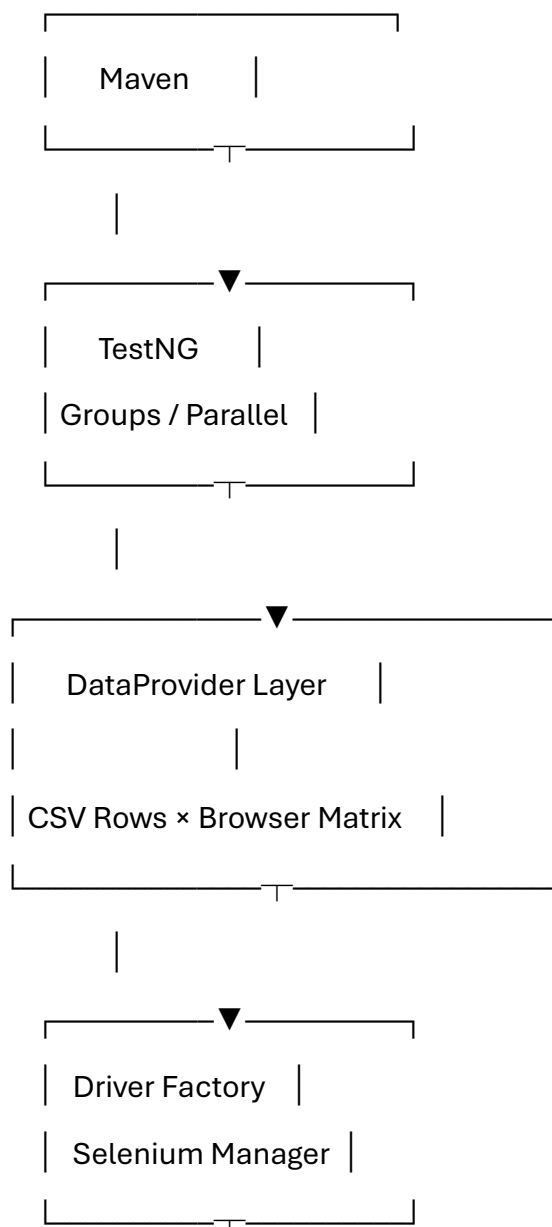
+

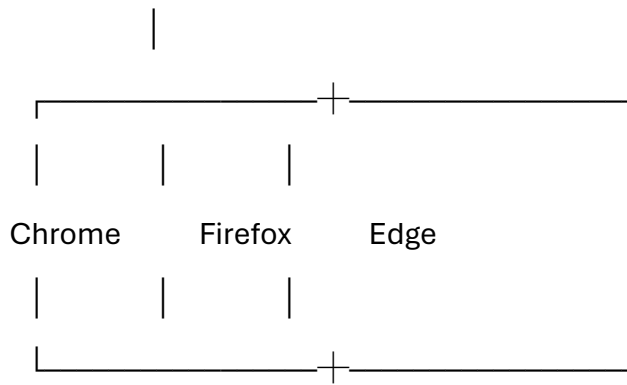
Test management integration

The Page Objects and test business logic can remain largely unchanged.

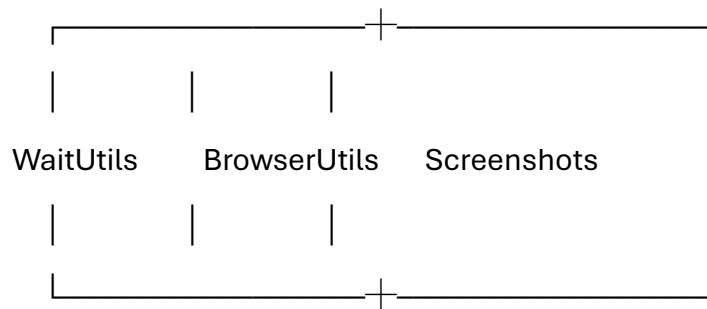
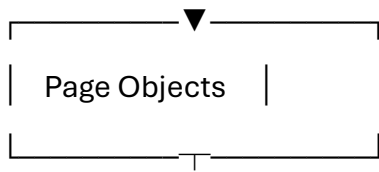
---

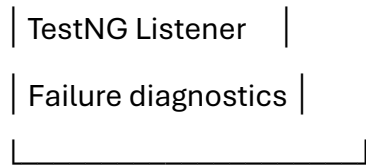
### Final architecture in one picture





ThreadLocal Driver





This gives you a **clean Selenium + Java + TestNG + Maven + Page Object + CSV data-driven framework** without introducing unnecessary enterprise complexity.

The next practical step would be to replace the placeholder LoginPage locators and CSV fields with your **actual application's first business flow**, then build the remaining Page Objects following exactly the same pattern.